Paths on Grids

SHSID CPC

Grid Paths I

Time limit: 1.00 s **Memory limit:** 512 MB

Consider an $n \times n$ grid whose squares may have traps. It is not allowed to move to a square with a trap.

Your task is to calculate the number of paths from the upper-left square to the lower-right square. You can only move right or down.

Input

The first input line has an integer n: the size of the grid.

After this, there are n lines that describe the grid. Each line has n characters: . denotes an empty cell, and $*$ denotes a trap.

Output

Print the number of paths modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 1000$

Example

Input:
```
4
....
.*..
...*
*...
```

Output:
```
3
```

$$\mathtt{dp}[x][y] = \begin{cases} \mathtt{dp}[x-1][y] + \mathtt{dp}[x][y-1] & \text{if } (x,y) \text{ is not a trap} \\ 0, & \text{if } (x,y) \text{ is a trap} \end{cases}$$

```cpp
#include <bits/stdc++.h>

using namespace std;

typedef long long ll;

bool ok[1000][1000];
ll dp[1000][1000];

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;
        for (int j = 0; j < n; j++) {
            if (s[j] == '.') ok[i][j] = true;
            else ok[i][j] = false;
        }
    }

    dp[0][0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (!ok[i][j]) dp[i][j] = 0;
            else {
                if (i > 0) dp[i][j] += dp[i - 1][j];
                if (j > 0) dp[i][j] += dp[i][j - 1];
                dp[i][j] %= 1000000007;
            }
        }
    }

    cout << dp[n - 1][n - 1] << "\n";

    return 0;
}
```

Longest Common Subsequence

Given A and B, find their longest common subsequence.

E.g. “DACE” and “ABCDE”, is “ACE”

Longest Common Subsequence

We notice that any subsequence can be constructed through a series of one of the following steps given pointer i and j starting at beginning

1. increase i by 1
2. increase j by 1
3. increase i and j together when Ai = Bj.

Longest Common Subsequence

$$\mathbf{dp}[i][j] = \begin{cases} \max(\mathbf{dp}[i-1][j], \mathbf{dp}[i][j-1]) & \text{if } A_i \neq B_j \\ \mathbf{dp}[i-1][j-1] + 1, & \text{if } A_i = B_j \end{cases}$$

	x	***a***	***b***	***c***	***d***
y	**0**	0	0	0	0
a	0	**1**	1	1	1
z	0	**1**	**1**	1	1
c	0	1	1	**2**	**2**

```
class Solution {
  public:
    int longestCommonSubsequence(string a, string b) {
        int dp[a.size()][b.size()];
        for (int i = 0; i < a.size(); i++) { fill(dp[i], dp[i] + b.size(), 0); }
        for (int i = 0; i < a.size(); i++) {
            if (a[i] == b[0]) dp[i][0] = 1;
            if (i != 0) dp[i][0] = max(dp[i][0], dp[i - 1][0]);
        }
        for (int i = 0; i < b.size(); i++) {
            if (a[0] == b[i]) dp[0][i] = 1;
            if (i != 0) dp[0][i] = max(dp[0][i], dp[0][i - 1]);
        }
        for (int i = 1; i < a.size(); i++) {
            for (int j = 1; j < b.size(); j++) {
                if (a[i] == b[j]) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[a.size() - 1][b.size() - 1];
    }
};
```

Edit Distance

Edit Distance

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

The *edit distance* between two strings is the minimum number of operations required to transform one string into the other.

The allowed operations are:

- Add one character to the string.
- Remove one character from the string.
- Replace one character in the string.

For example, the edit distance between LOVE and MOVIE is 2, because you can first replace L with M, and then add I.

Your task is to calculate the edit distance between two strings.

Input

The first input line has a string that contains n characters between A–Z.

The second input line has a string that contains m characters between A–Z.

Output

Print one integer: the edit distance between the strings.

Constraints

- $1 \leq n, m \leq 5000$

Example

Input:
```
LOVE
MOVIE
```

Output:
```
2
```

Edit Distance

Edit Distance

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

The *edit distance* between two strings is the minimum number of operations required to transform one string into the other.

The allowed operations are:

- Add one character to the string.
- Remove one character from the string.
- Replace one character in the string.

For example, the edit distance between LOVE and MOVIE is 2, because you can first replace L with M, and then add I.

Your task is to calculate the edit distance between two strings.

Input

The first input line has a string that contains n characters between A–Z.

The second input line has a string that contains m characters between A–Z.

Output

Print one integer: the edit distance between the strings.

Constraints

- $1 \leq n, m \leq 5000$

Example

Input:
```
LOVE
MOVIE
```

Output:
```
2
```

Let DP[i][j] denote th minimum distance to get the first i characters of the first word to the first j characters of the second.

DP[i][j] = ?

MIN:

if (Xi == Yj) DP[i-1][j-1]

else DP[i-1][j-1]+1

DP[i-1][j]

DP[i][j-1]

```java
import java.util.*;

public class EditString {
    public static void main(String[] args){
        Scanner reader = new Scanner(System.in);
        String str1 = ' ' + reader.next();
        String str2 = ' ' + reader.next();
        reader.close();
        int[][] dp = new int[str1.length()+1][str2.length()+1];
        for (int i = 0; i < dp.length; i++){
            Arrays.fill(dp[i], Integer.MAX_VALUE);
        }
        dp[0][0] = 0;
        for (int i = 0; i < str1.length(); i++){
            for (int j = 0; j < str2.length(); j++){
                if (i != 0)
                dp[i][j] = Math.min(dp[i][j], dp[i-1][j]+1);
                if (j != 0)
                dp[i][j] = Math.min(dp[i][j], dp[i][j-1]+1);
                if (i != 0 && j != 0)
                if (str1.charAt(i)==str2.charAt(j))
                dp[i][j] = Math.min(dp[i-1][j-1], dp[i][j]);
                else dp[i][j] = Math.min(dp[i-1][j-1]+1, dp[i][j]);
                // System.out.print(dp[i][j]);
            }
            // System.out.println();
        }
        System.out.println(dp[str1.length()-1][str2.length()-1]);
    }
}
```

Why Cross II

USACO 2017 February Contest, Gold
Problem 2. Why Did the Cow Cross the Road II

Return to Problem List

Contest has ended.

Log in to allow submissions in analysis mode

English (en)

Farmer John raises N breeds of cows ($1 \le N \le 1000$), conveniently numbered $1 \ldots N$. Some pairs of breeds are friendlier than others, a property that turns out to be easily characterized in terms of breed ID: breeds a and b are friendly if $|a - b| \le 4$, and unfriendly otherwise.

A long road runs through FJ's farm. There is a sequence of N fields on one side of the road (one designated for each breed), and a sequence of N fields on the other side of the road (also one for each breed). To help his cows cross the road safely, FJ wants to draw crosswalks over the road. Each crosswalk should connect a field on one side of the road to a field on the other side where the two fields have friendly breed IDs (it is fine for the cows to wander into fields for other breeds, as long as they are friendly). Each field can be accessible via at most one crosswalk (so crosswalks don't meet at their endpoints).

Given the ordering of N fields on both sides of the road through FJ's farm, please help FJ determine the maximum number of crosswalks he can draw over his road, such that no two intersect.

INPUT FORMAT (file nocross.in):

The first line of input contains N. The next N lines describe the order, by breed ID, of fields on one side of the road; each breed ID is an integer in the range $1 \ldots N$. The last N lines describe the order, by breed ID, of the fields on the other side of the road. Each breed ID appears exactly once in each ordering.

OUTPUT FORMAT (file nocross.out):

Please output the maximum number of disjoint "friendly crosswalks" Farmer John can draw across the road.

SAMPLE INPUT:

```
6
1
2
3
4
5
6
6
5
4
3
2
1
```

SAMPLE OUTPUT:

```
5
```

Problem credits: Brian Dean

Why Cross II

Let DP[i][j] denote th minimum distance to connect the first i on the left and first j on the right

DP[i][j] = min(DP[i-1][j], DP[i][j-1], if friendly(li, rj) DP[i-1][j-1]+1: DP[i-1][j-1]).

Minimum Grid Path

Minimal Grid Path

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

You are given an $n \times n$ grid whose each square contains a letter.

You should move from the upper-left square to the lower-right square. You can only move right or down.

What is the lexicographically minimal string you can construct?

Input

The first line has an integer n: the size of the grid.

After this, there are n lines that describe the grid. Each line has n letters between A and Z.

Output

Print the lexicographically minimal string.

Constraints

- $1 \leq n \leq 3000$

Example

Input:
```
4
AACA
BABC
ABDA
AACA
```

Output:
```
AAABACA
```

Minimum Grid Path

Let DP[i][j] denote the lexicographically minimum string to reach (i, j).

Then, DP[i][j] = min(DP[i-1][j], DP[i][j-1]) + c[i][j].

But that’s up to 3000*3000*6000 comparisons

Minimum Grid Path

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
char c[3005][3005];
bool chosen[3005][3005];
char seq[6005];
int main(){
    cin >> n;
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= n; j++){
            cin >> c[i][j];
        }
    }
    chosen[1][1] = 1;
    for (int i = 3; i <= 2*n; i++){
        char best = 'Z';
        for (int x = 1; x <= min(i, n); x++){
            int y = i-x;
            if (y > n) continue;
            if (chosen[x-1][y] || chosen[x][y-1]){
                if (c[x][y] < best) best = c[x][y];
            }
        }
        for (int x = 1; x <= min(i, n); x++){
            int y = i-x;
            if (y > n) continue;
            if (chosen[x-1][y] || chosen[x][y-1]){
                if (c[x][y] == best) chosen[x][y] = 1;
            }
        }
    }
    int x = n, y = n;
    for (int idx = 2*n-1; idx >= 1; idx--){
        seq[idx] = c[x][y];
        if (chosen[x-1][y]) x--;
        else y--;
    }
    for (int i = 1; i <= 2*n-1; i++){
        cout << seq[i];
    }
    cout << endl;
    return 0;
}
```

E - Swap. AtCoder

Problem Statement

Given is a string S consisting of K, E, Y.

How many strings are there that can be obtained with at most K swaps of two adjacent characters in S?

Constraints

- $2 \leq |S| \leq 30$
- $0 \leq K \leq 10^9$
- S consists of K, E, Y.

E - Swap. AtCoder

Firstly, the k is massive and scary. But notice how we can achieve every possible configuration within 30*29/2 = 435 swaps, by a bubble sort sorta idea.

Then, for a fixed final configuration, the way to optimize for the number of swaps is obvious: complete from left to right.

Let DP[cost][k][e][y] denote the number of ways to complete the first k+e+y with k Ks, e E's and y Y's, with cost cost.

Then, DP[cost + difference(k'th K position, k+e+y)][k][e][y]

Note that k'th K position may shift [covered in class]...